UNIVERSITY COURSE REGISTRATION & PREREQUISITES CHECKER

NAME: IRENE JINO

REG NO: 25BOE10004

INTRODUCTION

This project implements a University Course Registration and Prerequisite checker using Python. It is a computational tool designed to automate the enrolment process for students. In a modern academic environment, manually verifying if a student meets the criteria for a specific course is inefficient and prone to error. This program utilizes Python data structures—specifically dictionaries for course catalogues and lists for student records—to represent compound data effectively. By applying algorithm design techniques, the system evaluates academic eligibility in real-time.

PROBLEM STATEMENT

Given a set of available courses, each with specific credit values and prerequisite requirements, the task is to develop a program that:

- Identifies and abstracts the programming task of matching user input against a predefined course database.
- Validates enrolment by checking two primary constraints:
    1. Whether the student has completed the prerequisite courses.
    2. Whether adding the new course will cause the student to exceed the maximum allowed credit limit of 21 credits.
- Handles exceptions, such as when a student enters an invalid course code or attempts to register for a course they have already completed.

FUNCTIONAL REQUIREMENTS

- Input Validation: Accept a course code and convert it to uppercase to ensure standard string processing.
- Catalogue Check: Verify if the entered course code exists within the predefined course dictionary.
- Duplicate Prevention: Check the student's history to ensure they are not registering for a course they have already completed.
- Prerequisite Verification: Use conditional logic to confirm the student meets the specific "Nil" or named prerequisites for the chosen course.
- Credit Limit Enforcement: Calculate if adding the new course credits will exceed the maximum allowed limit of 21.
- Status Reporting: Output a clear "Success" message or a specific "Denied" error based on the logic results.

NON - FUNCTIONAL REQUIREMENTS

- Reliability: The system must consistently apply the same logic to every registration attempt, ensuring no errors in "Program Verification".
- Usability: The "Interactive Mode" must be intuitive enough for students to "effectively solve the task" without prior training.
- Performance: The "Time Trade-off" for searching the "Dictionary" and "Lists" must be minimal to provide instant feedback.
- Accuracy: The "Numerical Techniques" used for credit summation must be 100% accurate to prevent overloading a student's schedule.
- Robustness: The system must use "Logical Thinking" to handle "Invalid Input" (like non-existent course codes) without crashing.

SYSTEM ARCHITECTURE

The architecture follows a Top-Down Design and is structured into three layers to ensure the Efficiency of Algorithms:

- Data Layer: Uses Python Dictionaries to store course attributes (credits, prerequisites) and Lists to track student history.
- Logic Layer: Employs Conditional Statements (if-Elif-else) to validate course existence, prerequisites, and the 21-credit limit.
- Interface Layer: Operates in Interactive Mode to capture user input and display real-time "Success" or "Denied" status.

SCREENSHOTS/RESULTS

Code:

```python
# Project: Academic Enrollment System (Unit 5: Dictionaries & Lists)
def enroll_student():
    # Unit 5: Representing compound data [cite: 17]
    course_catalog = {
        "CSE1021": {"name": "Intro to Problem Solving", "credits": 4, "req": "Nil"},
        "MAT1011": {"name": "Calculus", "credits": 4, "req": "High School Math"},
        "CSE2001": {"name": "Data Structures", "credits": 3, "req": "CSE1021"},

        "ENG1001": {"name": "Effective Technical Communication", "credits": 2, "req": "Nil"}}

    # Unit 5: Student record
    student_history = ["High School Math"]
    current_credits = 18 # Max limit is 21

    course_code = input("Enter Course Code to Register: ").upper()

    # Unit 3: Conditional validation
    if course_code not in course_catalog:
        print("Error: Course code does not exist in the syllabus.")
    elif course_code in student_history:
        print("Error: You have already completed this course.")
    else:
        # Unit 3 & 5: Checking Prerequisites and Credit Limits
        required_pre = course_catalog[course_code]["req"]
        course_credits = course_catalog[course_code]["credits"]

        if required_pre != "Nil" and required_pre not in student_history:
            print(f"Registration Denied! You must complete {required_pre} first.")
        elif current_credits + course_credits > 21:
            print("Registration Denied! This would exceed your 21-credit limit.")
        elif course_credits >= 4:
            print(f"Success! Registered for {course_code}. Note: This is a high-credit core course.")
            current_credits += course_credits
        else:
            print(f"Success! Registered for {course_code}.")
            current_credits += course_credits

    print(f"Current Total Credits: {current_credits}")

enroll_student()
```

Output:

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

======= RESTART: C:\Users\Irene\OneDrive\Desktop\ffcs code for python.py =======
Enter Course Code to Register: CSE1021
Registration Denied! This would exceed your 21-credit limit.
Current Total Credits: 18

======= RESTART: C:\Users\Irene\OneDrive\Desktop\ffcs code for python.py =======
Enter Course Code to Register: CSE2001
Registration Denied! You must complete CSE1021 first.
Current Total Credits: 18

======= RESTART: C:\Users\Irene\OneDrive\Desktop\ffcs code for python.py =======
Enter Course Code to Register: ENG1001
Success! Registered for ENG1001.
Current Total Credits: 20

======= RESTART: C:\Users\Irene\OneDrive\Desktop\ffcs code for python.py =======
Enter Course Code to Register: MAT1011
Registration Denied! This would exceed your 21-credit limit.
Current Total Credits: 18
```

IMPORTANCE

This project is significant because it bridges the gap between manual academic rules and computational logic, addressing core syllabus objectives:

- Error Elimination: Manual checks are prone to oversight. This system ensures 100% accuracy in enforcing prerequisites and credit limits, which is vital for student academic success.
- Data Mastery: It provides a practical application for Unit 5's Compound Data. Using Dictionaries for fast lookups and Lists for dynamic history demonstrates how to organize real-world information efficiently.
- Logical Rigor: The project forces the use of the "Logic of Exclusion" via if-elif-else. By stopping a registration at the first failed check, it teaches robust Algorithm Design and "Program Verification."
- Constraint Management: It applies Numerical Techniques to manage a strict 21-credit boundary. This teaches how to implement safety constraints in software to prevent "overloading" a system (or a student).

CHALLENGES FACED

The challenges focused on the logical transition from a manual process to a robust Algorithm Design:

- Logic Sequencing: If prerequisites were checked before verifying if the course exists, the program will crash. I had to ensure a Top-Down order: Existence → Duplicate Check → Prerequisite → Credits.
- Data Accessing: Navigating "Compound Data" was tricky. I had to precisely extract the credits and values from a Nested Dictionary without affecting the rest of the data.
- Input Standardization: Python is case-sensitive. I resolved this by forcing all user input to uppercase, ensuring the "Program Verification" didn't fail due to simple typing differences.
- Predictive Validation: Instead of adding credits and checking after the fact, I implemented Numerical Logic to calculate the potential total first, preventing the student from ever exceeding the 21-credit limit.

LEARNINGS AND KEY TAKEAWAYS

- I learned that the order of Conditional is not random. To prevent system crashes, "Existence" must be verified before "Prerequisites."
- I mastered the use of Compound Data), specifically using Dictionaries for quick attribute lookups (credits) and Lists for tracking a student's growing history.
- A major takeaway was implementing Numerical Logic to calculate a potential total before updating the record. This ensures the 21-credit limit is never breached.
- I successfully applied Top-Down Design, proving that any complex real-world policy can be broken down into a simple, programmable flowchart.